

Controlled Language: Before and After

One document, two standards: commercial house style and ASD-STE100 Simplified Technical English

Roger Wong · rogerawong@gmail.com · rogerawong.github.io

I built this sample to demonstrate writing to a controlled-language specification. I shipped the original concept document with the CodeWarrior development tools at Metrowerks. This rewrite adapts it to ASD-STE100 Simplified Technical English (Issue 9), the controlled language of aerospace and defense maintenance documentation. Every difference you notice between the versions can be traced to an STE rule.

Three differences in thirty seconds

Before	After	The rule
"Caution! If debugging, compile your code without any optimizations."	"Note: If you optimize your code before you debug it, the debugger cannot connect the object code to the source code."	<i>Rule 7.1: a caution claims damage risk. This is information, so it demotes to a note; the commands move to body text (Rule 5.5).</i>
"Unreferenced virtual functions are never dead-stripped from code."	"The linker does not remove a virtual function when no code calls it."	<i>Rules 1.1 and 3.6: approved words, active voice, a named agent. The jargon becomes a sentence any reader can verify.</i>
"If you are not using MSL C++, normal inlining and judicious use of the inline keyword may reduce your code size."	"If you do not use MSL C++, the default inlining can make your code smaller. Use the inline keyword only on small functions."	<i>"May" is not approved vocabulary; possibility is "can." The -ing verb form goes (Rule 3.5); one idea per sentence.</i>

Declared technical names and technical verbs (STE rules 1.5, 1.8, 1.12)

STE permits words outside its dictionary only as declared technical names and technical verbs, kept in a governed word list. This sample declares, with one meaning and one part of speech each:

design (n): the plan and structure of a program

code size (n): the quantity of object code that a program makes

language feature (n): a capability that a programming language gives to the programmer

standard (n): a document of requirements, as in the ANSI/ISO standard for C++

Domain technical names: class, compiler, constructor, data section, data table, debugger, default, destructor, exception, getter, keyword, library, linker, operator, prefix file, program, RTTI, runtime, setter, setting, settings panel, stream, string, template specialization, vector. **Technical verbs:** build, call, debug, format, initialize, inline, optimize, parse, throw. **Code identifiers keep their exact forms:** cin, cout, cerr, sprintf, sscanf, inline, new, __throws_bad_alloc.

BEFORE

Commercial house style · Metrowerks, 2002

Concept documentation as shipped with the CodeWarrior development tools, lightly restored.

Smaller C++ Code for Size-Critical Programs

If size is a priority, several strategies can reduce the size of your object code at the expense of its execution speed.

Compiler-related Strategies

Most compiler-related strategies require only a settings change, not a source change.

Caution! If debugging, compile your code without any optimizations. Some optimizations break the relationship between source and object code required by the debugger. Optimize your code after debugging.

Enable Size Optimizations

The CodeWarrior compiler includes several levels of optimization for speed and size.

Select the Size optimization and set the Optimization Level in the Processor Settings panel.

Inlining May Help

Inlining small functions can make a program smaller, particularly if you have a class library with many getter/setter member functions.

For the smallest code size with Metrowerks Standard Library (MSL) C++, turn inlining off when you build the library. If you are not using MSL C++, normal inlining and judicious use of the `inline` keyword may reduce your code size.

Turn inlining on or off in the C/C++ Language settings panel.

Language-related Strategies

Language-related strategies limit or avoid ANSI/ISO C++ features that make software design and maintenance easier, but often do so at the expense of size.

Do Not Use Virtual Functions

Use virtual functions only when necessary. Unreferenced virtual functions are never dead-stripped from code.

Turn Off Runtime Type Identification (RTTI)

RTTI generates a data table for every class, so turning this off makes the data section smaller. Turn RTTI off in the C/C++ Language settings panel.

Turn Off Exception Handling

Use C++ exceptions selectively or not at all. The exception tables in the data section can get very large when exception handling is on.

Turn exception handling off in the C/C++ Language settings panel.

Avoid Exceptions in the New Operator

Some runtime libraries let the C++ `new` operator throw an exception. Disable this behavior in your prefix file by setting `__throws_bad_alloc` to 0.

Do Not Use Virtual Inheritance

Virtual base classes increase the size and complexity of the constructor and destructor functions.

Library-related Strategies

Library-related strategies reduce the amount of library code linked into your program.

Avoid Stream-Based Classes

Do not use stream-based classes when code size is critical. The MSL C++ stream classes initialize several objects, both directly and indirectly. Instead of using `cin`, `cout`, and `cerr`, use the standard C input and output functions.

Also avoid using string streams for formatting and parsing. Use standard C `sprintf` or `sscanf` functions instead.

Use an Alternative Library or Subset of Classes

MSL C++ has large overhead because each template specialization generates its own code. If you write your own vector, string, or other classes you commonly use, you can provide a subset of the standard library's functionality with less overhead.

You can also use an alternative class library that can suit your work better, such as the National Institutes of Health (NIH) Class Library. If you do use an alternative library, keep in mind possible problems with virtual inheritance, RTTI, or other causes of larger code size that we described above.

AFTER

ASD-STE100 Issue 9 · Simplified Technical English

The same document rewritten to STE writing rules, with every word checked against the Issue 9 dictionary.

Smaller C++ Code for Size-Critical Programs

You can make your object code smaller with the changes in this document. But most of these changes also make your program slower.

There are three types of changes: compiler settings, C++ language features, and libraries.

Compiler Settings

These changes use compiler settings. Usually, you do not change your source code.

Note: If you optimize your code before you debug it, the debugger cannot connect the object code to the source code.

Debug your code first. Then optimize it.

Select the Size Optimization

The CodeWarrior compiler has optimizations for speed and for smaller code size. Select the Size optimization in the Processor Settings panel. Then set the Optimization Level.

Use Inlining Carefully

When the compiler inlines small functions, your program can become smaller. The effect is largest for class libraries that have many small getter and setter functions.

If you use the Metrowerks Standard Library (MSL) for C++, set inlining to off when you build the library. This setting gives the smallest code. If you do not use MSL C++, the default inlining can make your code smaller. Use the `inline` keyword only on small functions.

Set inlining to on or off in the C/C++ Language settings panel.

C++ Language Features

Some ANSI/ISO C++ features make design and maintenance easier. But these features also make your code larger.

Do Not Use Virtual Functions

Use virtual functions only when your design makes them necessary. The linker does not remove a virtual function when no code calls it.

Set RTTI to Off

Runtime Type Identification (RTTI) makes a data table for each class. These tables make the data section larger. If your code does not use RTTI, set it to off in the C/C++ Language settings panel.

Set Exception Handling to Off

Use C++ exceptions only where they are necessary in your design. When exception handling is on, the compiler makes exception tables in the data section. These tables can become large. If your code does not use exceptions, set exception handling to off in the C/C++ Language settings panel.

Prevent Exceptions in the New Operator

In some runtime libraries, the C++ `new` operator can throw an exception. To prevent these exceptions, set `__throws_bad_alloc` to 0 in your prefix file.

Do Not Use Virtual Inheritance

Virtual base classes make the constructor and destructor functions larger. Do not use virtual base classes if they are not necessary in your design.

Libraries

These changes decrease the quantity of library code that the linker puts into your program.

Do Not Use Stream-Based Classes

The stream-based classes in MSL C++ make and initialize many objects. Do not use `cin`, `cout`, or `cerr`. Use the standard C input and output functions.

Do not use string streams to format or parse text. Use the standard C functions `sprintf` and `sscanf`.

Use a Different Library or Make New Classes

MSL C++ can make your object code much larger because it uses templates to agree with the ANSI/ISO standard for C++. Each template specialization makes new code the first time you use it.

To keep your object code small, write new classes for the functions that your program uses. For example, you can write a new vector class or a new string class.

You can also use a different class library that can be more applicable to your work, for example, the National Institutes of Health (NIH) Class Library. If you use a different class library, examine it first. Virtual inheritance, RTTI, and the other causes in this document can also make that code larger.